

Markov Chain Monte Carlo

Advanced Statistical Inference

Simone Rossi

Sampling from an unknown distribution

Problem: How to sample from a distribution $p(\theta \mid \mathbf{y})$ when we don't know its form and the normalization constant Z ?

...

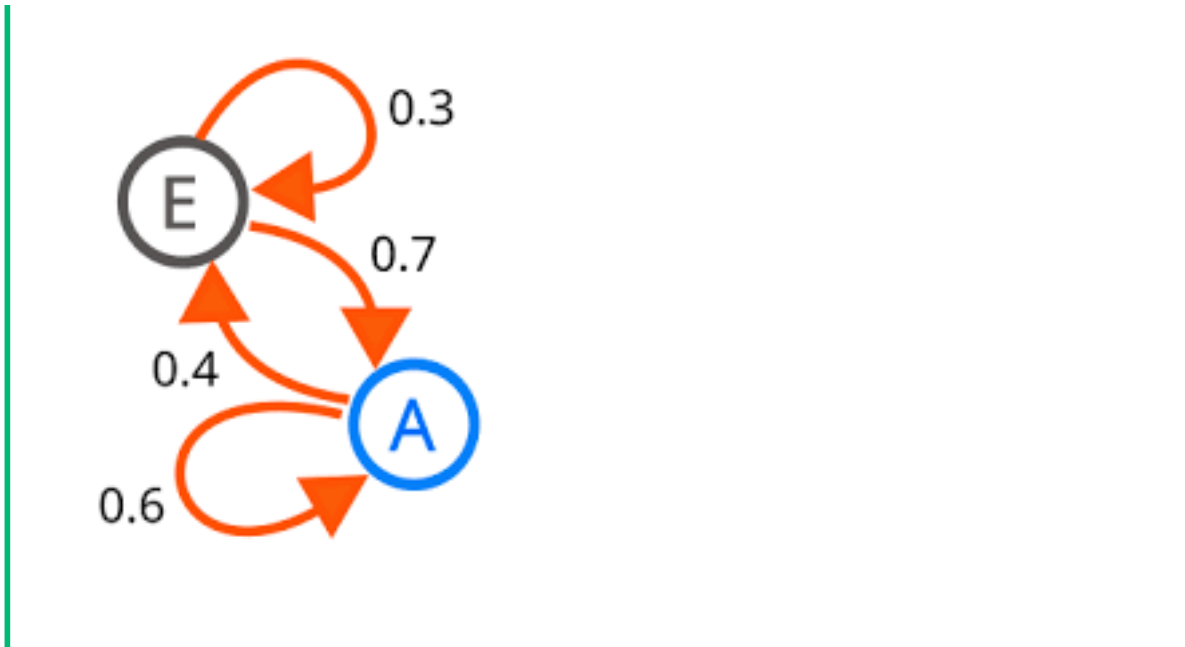
Markov Chain Monte Carlo (MCMC): use randomness to generate samples from a distribution in a way that the samples are correlated (Markov chain) and converge to the target distribution as the number of samples increases.

What is a Markov Chain?

A Markov chain is a sequence of random variables $X_1, X_2, \dots$ where the distribution of X_{n+1} depends only on X_n and not on the previous variables $X_1, \dots, X_{n-1}$.

It's described by:

- a transition probability $P(X_{n+1} \mid X_n)$ that defines the probability of transitioning from X_n to X_{n+1} .
- a **stationary distribution** p such that if $X_n \sim p$, then $X_{n+1} \sim p$ as well.



Metropolis-Hastings Algorithm

Objective: Build a Markov chain that has the target distribution $p(\theta \mid \mathbf{y})$ as its stationary distribution.

Solution: The Metropolis-Hastings (MH) algorithm is a specific MCMC method that constructs such a Markov chain.

...

MH algorithm produces a sequence of samples $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)}$ that converges to the target distribution $p(\theta \mid \mathbf{y})$.

Properties:

- **Markov Chain:** A sequence of random variables $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)}$ where the distribution of $\theta^{(t)}$ depends only on $\theta^{(t-1)}$.
- **Stationary Distribution:** The distribution of $\theta^{(t)}$ converges to the target distribution $p(\theta \mid \mathbf{y})$ as $t \rightarrow \infty$.

Metropolis-Hastings Algorithm

How to generate the sequence of samples $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(T)}$?

1. **Initialization:** Start with an initial value $\theta^{(1)}$.

2. **Proposal Distribution:** Generate a candidate sample θ' from a proposal distribution $q(\theta' | \theta^{(t)})$.
3. **Accepts/Rejects:** Accept the candidate sample based on some criteria.
 - If *accepted*, set $\theta^{(t+1)} = \theta'$.
 - If *rejected*, set $\theta^{(t+1)} = \theta^{(t)}$.
4. **Repeat:** Repeat steps 2-3 until we have T samples.

Metropolis-Hastings Algorithm: Proposal Distribution

$q(\theta' | \theta^{(t)})$ is a distribution that generates candidate samples θ' given the current sample $\theta^{(t)}$.

- We are free to choose any distribution as the proposal distribution (with the same or larger support as the target distribution).
- **Symmetric Proposal Distribution:** $q(\theta' | \theta^{(t)}) = q(\theta^{(t)} | \theta')$.
- **Random Walk Proposal:** $q(\theta' | \theta^{(t)}) = \mathcal{N}(\theta^{(t)}, \Sigma)$.

Metropolis-Hastings Algorithm: Acceptance Criteria

How to decide whether to accept or reject the candidate sample θ' ?

In the general case, acceptance based on:

$$\alpha = \frac{p(\theta' | \mathbf{y})q(\theta^{(t)} | \theta')}{p(\theta^{(t)} | \mathbf{y})q(\theta' | \theta^{(t)})}$$

If proposal distribution is symmetric,

$$\alpha = \frac{p(\theta' | \mathbf{y})}{p(\theta^{(t)} | \mathbf{y})} = \frac{p(\mathbf{y} | \theta')p(\theta')/Z}{p(\mathbf{y} | \theta^{(t)})p(\theta^{(t)})/Z} = \frac{p(\mathbf{y} | \theta')p(\theta')}{p(\mathbf{y} | \theta^{(t)})p(\theta^{(t)})}$$

Tip

Compute the acceptance ratio in the log-space to be numerically stable.

Metropolis-Hastings Algorithm: Acceptance Criteria

1. **Acceptance:** If $\alpha \geq 1$, accept the candidate sample θ' .
2. **Rejection:** If $\alpha < 1$, accept the candidate sample θ' with probability α .

Intuition:

1. If θ' is more likely than $\theta^{(t)}$, accept θ' .
2. If θ' is less likely than $\theta^{(t)}$, accept θ' with some probability.

Metropolis-Hastings Algorithm

Steps:

1. Initialize $\theta^{(1)}$.
2. For $t = 1, 2, \dots, T$:
 - Generate a candidate sample $\theta' \sim q(\theta' | \theta^{(t)})$.
 - Compute the acceptance ratio α .
 - Sample $u \sim \mathcal{U}(0, 1)$.
 - Compute $A = \min(1, \alpha)$.
 - Set new sample:

$$\theta^{(t+1)} = \begin{cases} \theta' & \text{if } u \leq A \text{ (accept)} \\ \theta^{(t)} & \text{if } u > A \text{ (reject)} \end{cases}$$

3. Repeat step 2 until we have T samples.

Why does Metropolis-Hastings work?

Detailed balance: The MH algorithm satisfies the detailed balance condition, which ensures that the Markov chain has the target distribution as its stationary distribution.

$$p(\theta^{(t)})P(\theta^{(t)} \rightarrow \theta') = p(\theta')P(\theta' \rightarrow \theta^{(t)})$$

Intuition (probability flow):

- So there is no net gain or loss of probability at any state.
- If the chain is distributed as p , one MH step leaves it distributed as p again (stationary distribution).

- The acceptance rule is exactly what corrects proposal bias: moves to high-density regions are accepted more often, and moves to low-density regions are accepted just enough to keep the two-way flows balanced.

Assessing Convergence

When to stop?

- MCMC algorithms converge to the target distribution as $T \rightarrow \infty$.
 - Practically, we need to decide when to stop the algorithm or to be able to assess its convergence.
1. **Burn-in Period:** Discard the initial samples to allow the chain to converge to the target distribution.
 2. **Visual Inspection:** Plot the trace of the samples and check for convergence.
 3. **Multiple Chains:** Run multiple chains from different initial values and compare their results.
 4. **Compute heuristics:** Compute some metrics to assess convergence.

Assessing Convergence with $\hat{R}$

Intuition: If multiple chains have converged, in-chain variance should be similar to between-chain variance.

Potential Scale Reduction Factor ($\hat{R}$):

$$\hat{R} = \sqrt{\frac{\text{between-chain variance}}{\text{in-chain variance}}}$$

In practice:

- $\hat{R} > 1.01$: Some chains have not converged, you may need to run the chains longer.
- $\hat{R} < 1.01$: Convergence has been reached.

Propose better and accept more

Hamiltonian Monte Carlo

Hamiltonian Monte Carlo (HMC): A more sophisticated MCMC algorithm that uses the concept of Hamiltonian dynamics to propose better samples.

Idea:

- Use gradient information to guide local exploration of the target distribution.
- Propose samples by simulating the dynamics of a particle moving in a potential energy landscape.

Hamiltonian Dynamics

A particle moving in a potential energy landscape $\mathcal{U}(\theta)$ with momentum ρ .

Hamiltonian:

$$\mathcal{H}(\theta, \rho) = \mathcal{U}(\theta) + \mathcal{K}(\rho)$$

- $\mathcal{U}(\theta)$: Potential energy of the particle at position θ .
- $\mathcal{K}(\rho)$: Kinetic energy of the particle with momentum ρ .

We consider:

- $\mathcal{U}(\theta) = -\log p(\theta | \mathbf{y})$, where $p(\theta | \mathbf{y})$ is the target distribution.
- $\mathcal{K}(\rho) = \frac{1}{2}\rho\mathbf{M}^{-1}\rho$, where $\mathbf{M}$ is the mass matrix.

Energy Conservation

- In case of no-friction, the Hamiltonian is conserved.
- The trajectory of the particle in the potential energy landscape is governed by the **Hamiltonian equations**:

$$\begin{aligned}\frac{d\boldsymbol{\theta}}{dt} &= \nabla_{\boldsymbol{\rho}} \mathcal{H}(\boldsymbol{\theta}, \boldsymbol{\rho}) = \nabla_{\boldsymbol{\rho}} \mathcal{K}(\boldsymbol{\rho}) \\ \frac{d\boldsymbol{\rho}}{dt} &= -\nabla_{\boldsymbol{\theta}} \mathcal{H}(\boldsymbol{\theta}, \boldsymbol{\rho}) = -\nabla_{\boldsymbol{\theta}} \mathcal{U}(\boldsymbol{\theta})\end{aligned}$$

The energy is conserved: $\frac{d\mathcal{H}}{dt} = 0$.

! Important

To simulate the dynamics of the particle, we don't need to know the normalization constant Z for $\mathcal{U}(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta} \mid \mathbf{y})$.

Why? Because the dynamics only depend on the gradient of the log-density $\nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta} \mid \mathbf{y}) = \nabla_{\boldsymbol{\theta}} [\log p(\mathbf{y} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) - \log Z]$.

Numerical Solution to the Hamiltonian Equations

$$\begin{aligned}\frac{d\boldsymbol{\theta}}{dt} &= \nabla_{\boldsymbol{\rho}} \mathcal{K}(\boldsymbol{\rho}) \\ \frac{d\boldsymbol{\rho}}{dt} &= -\nabla_{\boldsymbol{\theta}} \mathcal{U}(\boldsymbol{\theta})\end{aligned}$$

Solving exactly is infeasible, but we can use numerical methods to approximate the solution.

Leapfrog Integrator:

Start with $\boldsymbol{\theta}, \boldsymbol{\rho}$.

Repeat for L steps:

1. Update momentum: $\boldsymbol{\rho} \leftarrow \boldsymbol{\rho} - \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \mathcal{U}(\boldsymbol{\theta})$.
2. Update position: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \nabla_{\boldsymbol{\rho}} \mathcal{K}(\boldsymbol{\rho})$.
3. Update momentum: $\boldsymbol{\rho} \leftarrow \boldsymbol{\rho} - \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \mathcal{U}(\boldsymbol{\theta})$.

Simulating Hamiltonian Dynamics $\Leftrightarrow$ Sampling

$$p(\boldsymbol{\theta}, \boldsymbol{\rho}) = \frac{1}{Z_H} \exp(-\mathcal{H}(\boldsymbol{\theta}, \boldsymbol{\rho})) = \frac{1}{Z_H} \exp\left(-\mathcal{U}(\boldsymbol{\theta}) - \frac{1}{2} \boldsymbol{\rho}^\top \mathbf{M}^{-1} \boldsymbol{\rho}\right)$$

The marginal distribution of $\boldsymbol{\theta}$ is the target distribution $p(\boldsymbol{\theta} \mid \mathbf{y})$.

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \int p(\boldsymbol{\theta}, \boldsymbol{\rho}) d\boldsymbol{\rho} = \frac{1}{Z} \exp(-\mathcal{U}(\boldsymbol{\theta})) \frac{1}{Z_K} \int \exp\left(-\frac{1}{2} \boldsymbol{\rho}^\top \mathbf{M}^{-1} \boldsymbol{\rho}\right) d\boldsymbol{\rho} = \frac{1}{Z} \exp(-\mathcal{U}(\boldsymbol{\theta}))$$

Hamiltonian Monte Carlo Algorithm

Suppose the last sample in the sequence is $\theta^{(t-1)}$

1. Initialization:

- Set $\theta'_0 = \theta^{(t-1)}$.
- Sample momentum $\rho'_0 \sim \mathcal{N}(0, \mathbf{M})$.

2. Hamiltonian Dynamics:

- Simulate Hamiltonian dynamics for L steps to obtain θ'_L, ρ'_L .

3. Acceptance:

- Accept with probability α

$$\alpha = \min \left(1, \frac{p(\theta'_L, \rho'_L)}{p(\theta^{(t-1)}, \rho^{(t-1)})} \right) = \min \left(1, \exp \left(-\mathcal{H}(\theta'_L, \rho'_L) + \mathcal{H}(\theta^{(t-1)}, \rho^{(t-1)}) \right) \right)$$

Tuning HMC

- **Number of Steps:** The number of steps L to simulate the Hamiltonian dynamics.
- **Step Size:** The size of the leapfrog steps ϵ .
- **Mass Matrix:** The mass matrix $\mathbf{M}$.

Considerations:

1. L needs to be large enough to explore the target distribution, but small enough to avoid wasting computation.
2. ϵ needs to be small enough to avoid numerical instability, but large enough to have α close to 1.
3. $\mathbf{M}$ can be set to the identity matrix or estimated from the target distribution (after burn-in).

Extensions of HMC

- **No-U-Turn Sampler (NUTS):** Automatically adapt the number of steps L .
- **Stochastic Gradient Hamiltonian Monte Carlo:** Use stochastic gradients to scale to large datasets

$$\mathcal{U}(\theta) = -\log p(\theta \mid \mathbf{y}) \propto -\sum_{i=1}^n \log p(\mathbf{y}_i \mid \theta) - \log p(\theta)$$